

ESPECIFICACIÓN DE REQUERIMIENTOS DE SOFTWARE

Sistema de Gestión de Horarios Académicos

Facultad de Ingeniería y Tecnologías (FIT)

Universidad Católica del Uruguay

Versión 1.3

4 de marzo de 2026

Índice

1. Introducción.....	3
1.1 Propósito.....	3
1.2 Alcance.....	4
1.3 Contexto y problemática.....	4
2. Arquitectura del sistema.....	5
2.1 Visión general.....	5
2.2 Principios arquitectónicos.....	7
3. Stack tecnológico.....	8
3.1 Electron.....	8
3.2 React.....	9
3.3 SQLite.....	9
3.4 Restricciones e integridad de datos.....	9
4. Estructura del proyecto.....	10
4.1 Carpeta electron/.....	10
4.2 Carpeta front/.....	11
4.3 Carpeta db/.....	11
4.4 Carpeta modules/.....	12
4.5 Carpeta scripts/.....	12
4.6 Archivos de configuración principales.....	12
4.7 Organización general y coherencia estructural.....	12
5. Modelo de datos.....	13
5.1 Modelo Entidad-Relación.....	13
5.2 Diagrama físico.....	14
6. Flujo del sistema.....	14
6.1 Objetivo del flujo.....	14
7. Reglas de negocio.....	15
7.1 Reglas sobre profesores.....	15
7.2 Reglas sobre grupos.....	15
7.3 Reglas sobre horarios.....	15
7.4 Reglas sobre exportación de excel.....	16
7.5 Reglas sobre importación de excel.....	16
7.6 Reglas sobre materias.....	16
7.7 Reglas sobre carreras.....	16
7.8 Reglas de integridad.....	16
8. Interfaces / Pantallas.....	17
8.1 Pantalla principal.....	17
8.2 Modal de creación.....	18
8.3 Modal de edición.....	19

9. Instalación y ejecución.....	20
10. Limitaciones actuales.....	21
10.1 Uso monousuario.....	21
10.2 Base de datos local.....	21
10.3 Ausencia de sistema de roles.....	21
10.4 Escalabilidad institucional.....	21
10.5 Gestión avanzada de conflictos.....	21
11. Futuras mejoras.....	21
11.1 Aplicación multiusuario.....	21
11.2 Base de datos centralizada.....	22
11.3 Auditoría.....	22
11.4 Expandir a todas las facultades.....	23
11.5 IA para recomendaciones de horarios.....	23

1. Introducción

1.1 Propósito

El presente documento describe la arquitectura, decisiones técnicas y funcionamiento general de CalendarioFIT, desarrollado como solución tecnológica para la gestión y planificación de horarios académicos de la Facultad de Ingeniería y Tecnologías.

Esta especificación cumple un doble objetivo. Por un lado, documentar formalmente la arquitectura, reglas de negocio, decisiones técnicas y funcionamiento general del

sistema. Por otro lado, sirve como guía para futuros equipos de desarrollo que deban mantener, ampliar o migrar la aplicación.

La aplicación busca resolver una necesidad concreta mediante una solución desktop portable, permitiendo crear, modificar y mantener de forma centralizada los horarios de todas las carreras de la facultad, eliminando inconsistencias y facilitando la planificación académica.

1.2 Alcance

CalendarioFIT es una aplicación de escritorio diseñada para centralizar la planificación académica semestral. El sistema permite administrar carreras, materias, docentes, grupos y horarios, integrando en una única plataforma la información que anteriormente se encontraba distribuida en múltiples archivos y herramientas.

El sistema contempla la creación y edición de grupos académicos, incluyendo la posibilidad de asignar docentes o dejar la asignación pendiente durante las etapas iniciales de planificación. Asimismo, permite definir horarios por día y módulo, respetando la estructura de turnos oficiales de la universidad.

Entre sus capacidades se encuentra la detección automática de conflictos de asignación, particularmente en casos de superposición de docentes en un mismo módulo horario. También permite la importación estructurada de información desde archivos Excel y la exportación de calendarios consolidados en formato Excel para su distribución institucional.

El sistema fue diseñado bajo el supuesto de uso monousuario y funcionamiento completamente offline, sin requerir servidor externo ni infraestructura adicional. Esta decisión responde a los requisitos de portabilidad y simplicidad operativa establecidos al inicio del proyecto.

1.3 Contexto y problemática

La planificación académica dentro de la facultad presentaba diversos desafíos estructurales. En particular, la utilización de hojas de cálculo extensas y no centralizadas generaba inconsistencias cuando una materia compartida por varias

carreras era modificada en un único plan, sin replicarse adecuadamente en los demás.

Asimismo, la asignación manual de docentes y módulos incrementan el riesgo de superposición de horarios, generando conflictos que debían ser detectados y corregidos manualmente. La ausencia de una herramienta centralizada dificulta la visualización global del calendario académico, obligando a los coordinadores a realizar verificaciones cruzadas entre múltiples documentos.

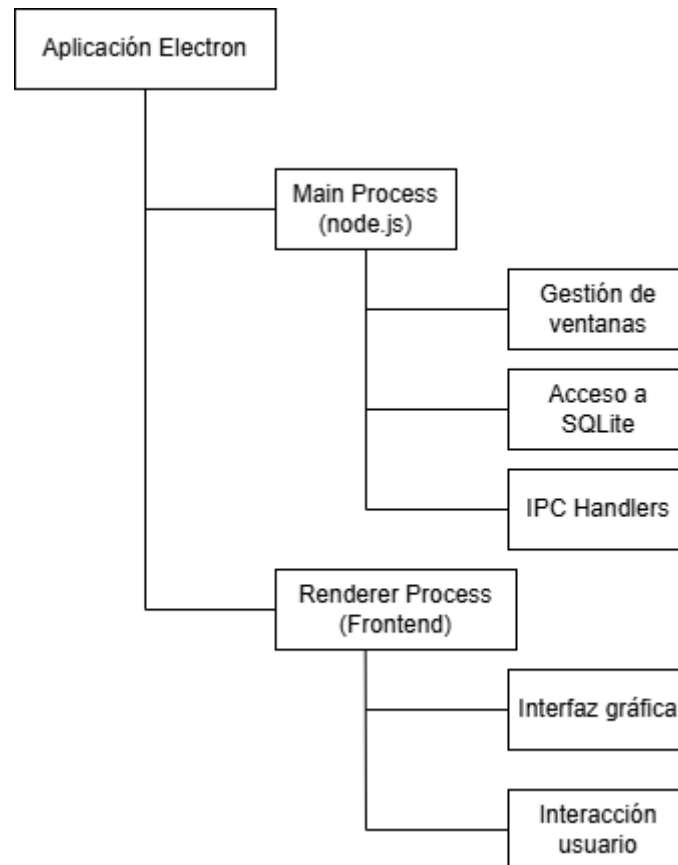
En este contexto, se identificó la necesidad de desarrollar una solución que permitiera unificar la información, reducir la probabilidad de errores humanos y automatizar las validaciones más críticas del proceso de planificación.

CalendarioFIT surge como respuesta a esta problemática, proponiendo una herramienta integrada, validada y estructurada.

2. Arquitectura del sistema

2.1 Visión general

El sistema sigue una arquitectura desacoplada basada en Electron:



El sistema CalendarioFIT fue diseñado siguiendo una arquitectura desacoplada basada en Electron, con una clara separación entre la capa de presentación, la lógica de negocio y la capa de persistencia. Esta decisión responde a la necesidad de mantener un diseño estructurado, mantenible y preparado para futuras extensiones.

La aplicación se ejecuta como un software de escritorio multiplataforma, utilizando Electron como contenedor principal. Internamente, la arquitectura se compone de cuatro elementos fundamentales: la interfaz desarrollada en React, el mecanismo de comunicación entre procesos (IPC), el proceso principal de Electron (Main Process) y la base de datos SQLite.

La interfaz de usuario se ejecuta en el proceso de renderizado (Renderer Process) y es responsable exclusivamente de la presentación visual y la interacción con el usuario. En esta capa no se implementa acceso directo a la base de datos ni lógica de negocio crítica.

La lógica de negocio se centraliza en el proceso principal (Main Process), el cual actúa como intermediario entre la interfaz y la base de datos. Todas las

validaciones, reglas de negocio y operaciones sensibles se ejecutan en esta capa, garantizando que la persistencia y las restricciones del sistema no dependan de la interfaz gráfica.

La persistencia de datos se gestiona mediante SQLite, una base de datos embebida que permite almacenar la información en un único archivo físico, eliminando la necesidad de infraestructura externa.

Esta arquitectura desacoplada permite mantener una separación clara de responsabilidades, reduciendo el acoplamiento entre componentes y mejorando la mantenibilidad del sistema.

2.2 Principios arquitectónicos

El diseño del sistema se apoya en una serie de principios arquitectónicos que orientaron las decisiones técnicas adoptadas durante el desarrollo.

En primer lugar, se aplicó el principio de separación de responsabilidades. La interfaz de usuario se limita a la representación visual y a la captura de eventos del usuario, mientras que la lógica de negocio y las operaciones sobre la base de datos se encuentran completamente aisladas en el proceso principal. Esto evita la mezcla de código de presentación con lógica crítica y facilita la evolución independiente de cada capa.

En segundo lugar, se implementó un modelo de comunicación controlada entre procesos utilizando el mecanismo de Inter-Process Communication (IPC) provisto por Electron. La interfaz React no accede directamente a la base de datos ni a las APIs de Node.js. En su lugar, toda solicitud se canaliza mediante el siguiente flujo estructurado:

Interfaz (Renderer) → Preload → IPC → Main Process → SQLite

El archivo preload actúa como capa de integración segura, exponiendo únicamente las funciones necesarias al entorno del Renderer. De esta forma, se evita la exposición directa de funcionalidades sensibles y se refuerza el aislamiento entre la interfaz y el entorno Node.

Finalmente, se priorizó la mantenibilidad y la posibilidad de evolución futura. La arquitectura adoptada permite, en caso de ser necesario, migrar la capa de persistencia hacia un modelo cliente-servidor o incorporar autenticación multiusuario sin necesidad de rediseñar completamente la estructura del sistema.

En conjunto, estas decisiones permiten que CalendarioFIT mantenga una estructura clara, segura y preparada para su extensión futura.

3. Stack tecnológico

El desarrollo de CalendarioFIT se apoyó en un conjunto de tecnologías seleccionadas en función de los requerimientos iniciales del proyecto, particularmente aquellos relacionados con portabilidad, simplicidad de despliegue, mantenibilidad del código y robustez estructural.

3.1 Electron

Electron fue elegido como entorno de ejecución para construir una aplicación de escritorio multiplataforma utilizando tecnologías web. Esta decisión permitió desarrollar el sistema con JavaScript y ejecutarlo en Windows, Linux o macOS sin necesidad de servidor externo.

Una de las principales ventajas de Electron es su modelo de procesos diferenciados. El sistema se estructura en:

- Main Process, encargado de la lógica de negocio, gestión de ventanas y acceso a la base de datos.
- Renderer Process, donde se ejecuta la interfaz gráfica desarrollada en React.

La comunicación entre ambos se realiza mediante IPC (Inter-Process Communication), garantizando que la interfaz no acceda directamente a la base de datos. Esta separación mejora la seguridad, reduce el acoplamiento entre capas y facilita la evolución futura del sistema.

Además, Electron permite empaquetar la aplicación junto con su base de datos embebida, cumpliendo con el requisito de portabilidad total.

3.2 React

React fue seleccionado para el desarrollo de la interfaz gráfica debido a su enfoque basado en componentes y su capacidad para construir interfaces dinámicas y mantenibles.

El sistema requiere:

- Formularios con validaciones.
- Representación visual de calendarios por días y módulos.
- Actualización reactiva ante cambios de estado.

React permite estructurar la interfaz en componentes reutilizables, separar lógica de presentación y mantener un estado claro y controlado. Esto favorece la mantenibilidad del código y facilita la incorporación de nuevas funcionalidades sin afectar al resto del sistema.

3.3 SQLite

Para la persistencia de datos se optó por SQLite como base de datos relacional embebida. Esta elección responde al requisito de funcionamiento offline y ausencia de infraestructura adicional.

SQLite opera mediante un único archivo físico, lo que simplifica la instalación y distribución del sistema. No requiere configuración de servidor ni servicios adicionales, lo cual reduce la complejidad operativa.

Dado que el sistema está diseñado inicialmente para un entorno monousuario, SQLite representa una solución adecuada en términos de rendimiento, simplicidad y mantenimiento.

3.4 Restricciones e integridad de datos

La integridad del sistema no depende únicamente de validaciones en la interfaz, sino que se refuerza mediante restricciones estructurales en la base de datos.

Se implementaron:

- Claves primarias.

- Claves foráneas.
- Restricciones de unicidad.
- Reglas de integridad referencial.

Estas restricciones previenen inconsistencias como asociaciones inválidas o duplicación de registros críticos.

Complementariamente, las reglas de negocio más complejas —como la detección de superposición horaria de docentes— se implementan en la capa del Main Process, garantizando un doble nivel de control: estructural y lógico.

4. Estructura del proyecto

El repositorio de CalendarioFIT se encuentra organizado en múltiples carpetas que reflejan la separación de responsabilidades definida en la arquitectura del sistema. Esta organización permite distinguir claramente la capa de presentación, la lógica principal de la aplicación, la persistencia de datos y los recursos auxiliares.

A continuación se describe la función de cada directorio y archivo principal del proyecto.

4.1 Carpeta `electron/`

Esta carpeta contiene la configuración y el código asociado al proceso principal (Main Process) de Electron.

En esta sección se encuentran:

- La inicialización de la aplicación.
- La creación y configuración de la ventana principal.
- La definición del archivo preload.
- La configuración de seguridad (contextIsolation, nodeIntegration, etc.).
- La definición de los manejadores IPC que conectan la interfaz con la lógica de negocio.

El código dentro de esta carpeta es responsable de coordinar la ejecución del sistema y actuar como intermediario entre la interfaz React y la base de datos SQLite.

4.2 Carpeta front/

La carpeta front contiene la aplicación desarrollada en React, correspondiente al Renderer Process de Electron.

Aquí se encuentran:

- Componentes visuales.
- Formularios de creación y edición.
- Pantalla principal con el calendario.
- Manejo del estado de la aplicación.
- Lógica asociada a la interacción del usuario.

Esta capa no accede directamente a la base de datos ni contiene lógica de negocio estructural. Todas las operaciones que implican persistencia se realizan mediante llamadas IPC hacia el proceso principal.

4.3 Carpeta db/

La carpeta db concentra todo lo relacionado con la persistencia de datos.

Incluye:

- Definición del esquema de base de datos.
- Configuración de Drizzle ORM.
- Posibles scripts de inicialización (seed).
- Conexión a SQLite.

Esta organización permite centralizar la definición de tablas, relaciones y restricciones en un único lugar del proyecto, facilitando el mantenimiento del modelo de datos.

4.4 Carpeta `modules/`

La carpeta `modules` contiene módulos de lógica reutilizable del sistema. En esta sección se agrupan funcionalidades que no pertenecen estrictamente a la interfaz ni a la configuración principal, pero que encapsulan lógica específica del dominio.

Esta estructura favorece la modularidad y permite aislar funcionalidades particulares del resto del código.

4.5 Carpeta `scripts/`

La carpeta `scripts` contiene utilidades auxiliares utilizadas durante el desarrollo, como posibles scripts de inicialización de base de datos, migraciones o automatización de tareas.

Este tipo de separación mejora la claridad del proyecto al evitar mezclar herramientas de soporte con la lógica principal de la aplicación.

4.6 Archivos de configuración principales

En la raíz del proyecto se encuentran archivos clave para la configuración general:

- `package.json`: define las dependencias del proyecto, los scripts de ejecución (`npm run start`) y la configuración base del entorno.
- `drizzle.config.js`: archivo de configuración del ORM utilizado para definir y sincronizar el esquema de base de datos.
- `.gitignore`: define los archivos excluidos del control de versiones.
- `README.md`: contiene instrucciones básicas de ejecución y descripción general del proyecto.

4.7 Organización general y coherencia estructural

La estructura del repositorio refleja una intención clara de separar:

- Configuración del entorno.
- Lógica principal de Electron.
- Interfaz desarrollada en React.

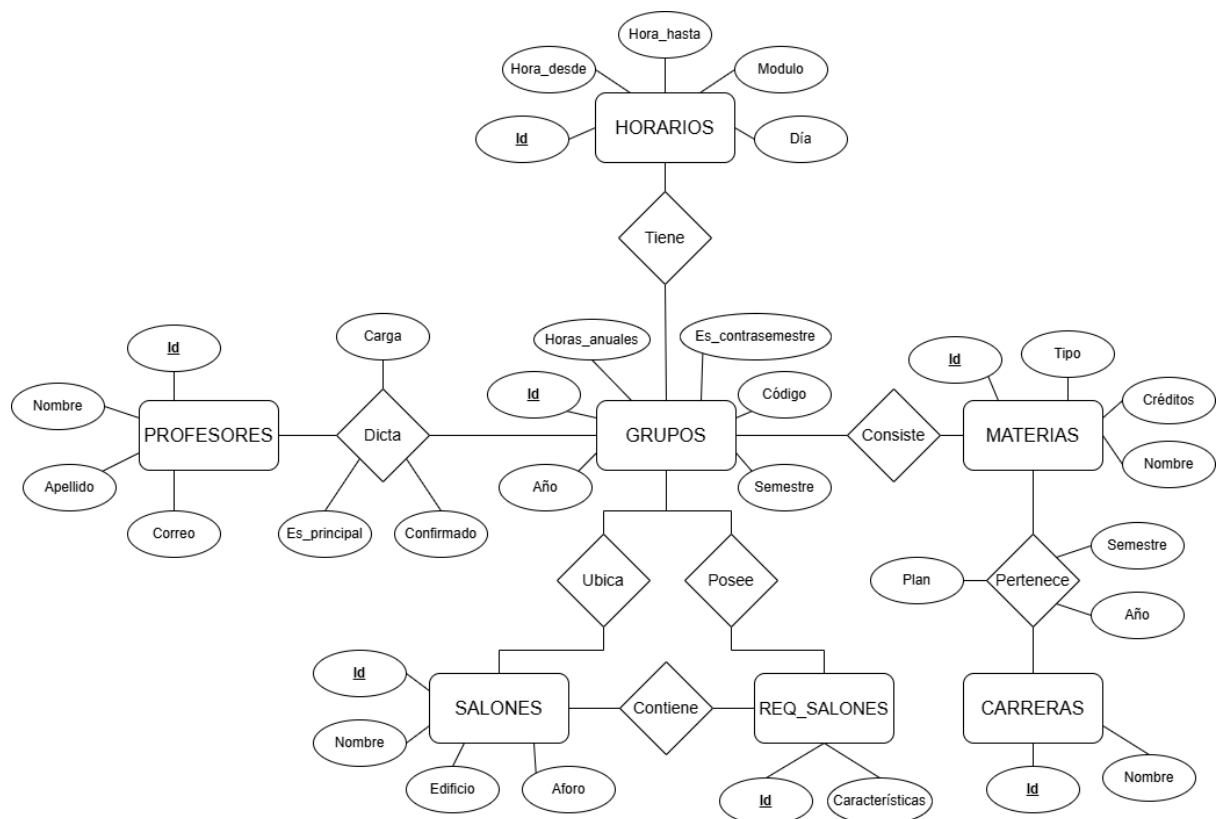
- Modelo de datos y persistencia.
- Recursos externos de importación/exportación.

Esta organización permite que futuros desarrolladores identifiquen rápidamente dónde realizar modificaciones específicas y reduce la probabilidad de acoplamiento innecesario entre capas.

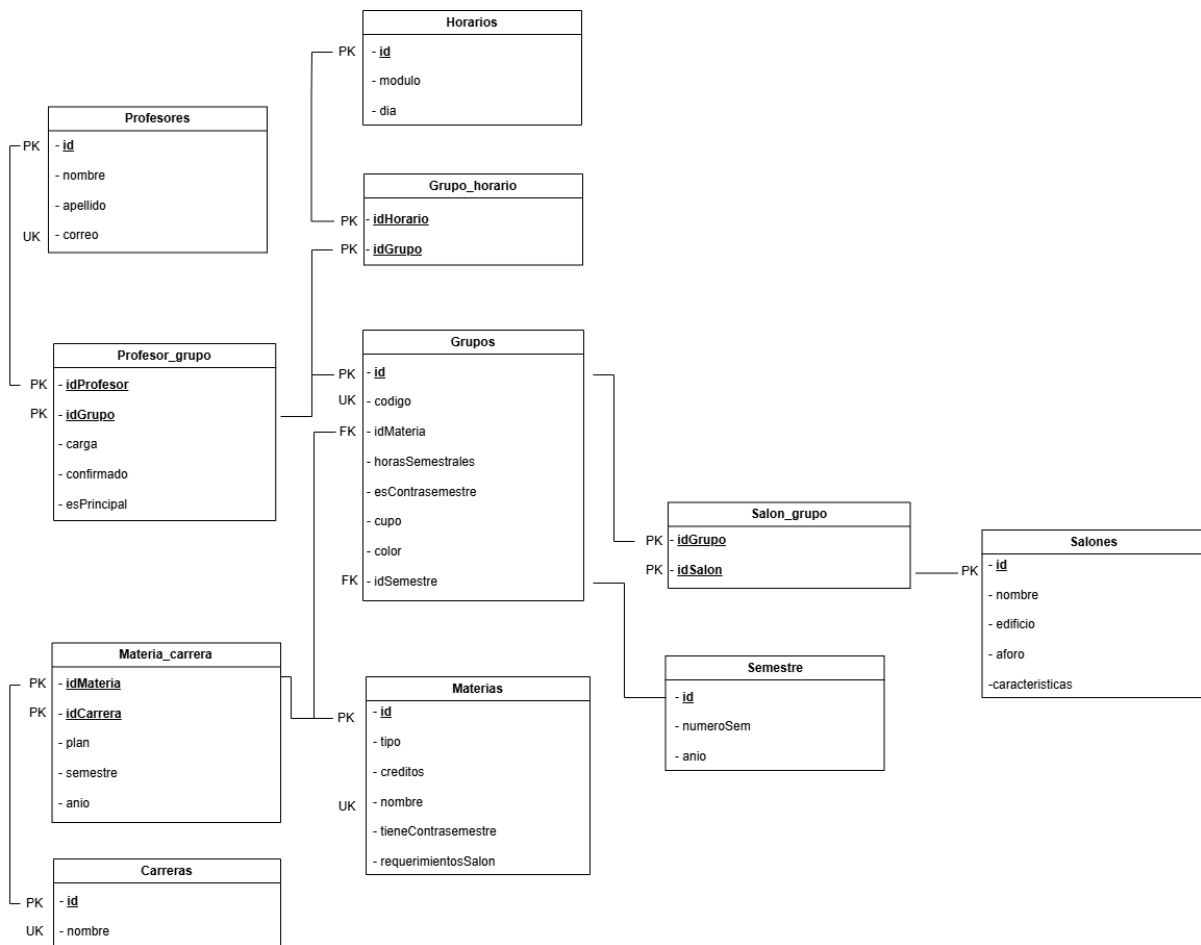
Asimismo, la estructura mantiene coherencia con la arquitectura descrita en el punto anterior, respetando los principios de separación de responsabilidades y modularidad.

5. Modelo de datos

5.1 Modelo Entidad-Relación



5.2 Diagrama físico



6. Flujo del sistema

6.1 Objetivo del flujo

El objetivo del flujo es garantizar una separación clara entre:

- Interfaz (React / Renderer): presentación y experiencia de usuario.
- Capa de integración segura (Preload): “puente” controlado hacia Electron.
- Lógica de negocio (Main Process): validaciones, reglas y coordinación.
- Persistencia (SQLite): almacenamiento local y consultas.

Esta separación mejora la seguridad (React no accede directo a Node/SQLite), facilita el mantenimiento y permite escalar la aplicación sin mezclar UI con base de datos.

7. Reglas de negocio

Dentro de cada clase y relación se encuentran distintas reglas de negocio que generan restricciones que deben ser cumplidas de forma obligatoria, se enumeran a continuación:

7.1 Reglas sobre profesores

- Un profesor no puede estar asignado a dos grupos en el mismo día y mismo módulo.
- Un profesor puede estar asignado a múltiples grupos mientras no haya superposición de horario
- Un profesor puede estar asignado a un grupo sin estar confirmado

7.2 Reglas sobre grupos

- Cada grupo debe estar asociado obligatoriamente a un año, semestre y materia
- Un grupo puede estar asociado a una o múltiples carreras (grupos compartidos)
- No puede existir más de un grupo con el mismo identificador dentro del mismo año y semestre.
- Un grupo puede crearse sin tener profesores asignados (TBD)
- Un grupo puede crearse sin código asignado
- Un grupo puede tener múltiples profesores
- Un grupo debe tener obligatoriamente un profesor principal (profesor atributo `es_principal=true` o TBD)

7.3 Reglas sobre horarios

- Los turnos de los horarios se respetan con los aplicados por la universidad.
- El horario se define por módulo y día

- No puede existir superposición de horarios para un mismo profesor
- Un grupo debe tener al menos un horario

7.4 Reglas sobre exportación de excel

- El excel a exportar debe contener: carrera, año, semestre, grupo, materia, horario y profesor, preferiblemente en formato calendario.
- Solo se deben exportar grupos válidos

7.5 Reglas sobre importación de excel

- El excel a importar debe tener los datos estructurados, pudiendo importar columnas nuevas como “código grupo”
- Las columnas obligatorias deben estar presentes
- Si el archivo contiene inconsistencias se debe abortar o generar un reporte

7.6 Reglas sobre materias

- Una materia puede estar asociada a múltiples carreras
- Si el atributo tiene_contrasemestre es true, la materia puede incluirse en el calendario de contrasemestre

7.7 Reglas sobre carreras

- No pueden existir dos carreras con el mismo nombre
- Una carrera debe tener múltiples materias asociadas
- Si se elimina una carrera, se eliminan las asociaciones con las materias

7.8 Reglas de integridad

- No puede existir un grupo sin una materia asociada
- No puede existir la asociación grupo-profesor sin que ambos existan
- No puede existir la asociación grupo-horario sin que ambos existan

8. Interfaces / Pantallas

La aplicación define una pantalla principal que centraliza todas las funcionalidades del sistema, permitiendo al usuario acceder de forma directa a la gestión de las distintas entidades.

El resto de interfaces son ventanas formato modal, que son utilizadas para creación, modificación y eliminación de las distintas entidades del sistema, las cuales son:

- Carreras
- Asignaturas
- Docentes
- Grupos
- Semestre

Cada modal permite diferentes acciones dentro de las que se destacan:

- Ingreso de los datos obligatorios
- Validación automática de las reglas de negocio
- Confirmación y cancelación de la operación
- Visualización de errores en caso de inconsistencias

8.1 Pantalla principal

La pantalla principal muestra el calendario con los grupos creados en los turnos correspondientes, se muestra un ejemplo a continuación:

← Volver Crear docente X

Nombre

Ej: Ana

Apellido

Ej: Gomez

Correo

ejemplo@ucu.edu.uy

Confirmar

Este modal permite crear un docente, como se puede observar piden los datos obligatorios necesarios (nombre, apellido y correo). Esto funciona igual para las otras entidades del sistema (carrera, asignatura, grupo, etc).

8.3 Modal de edición

← Volver Editar asignatura X

Nombre de la asignatura

Programacion 1

Tipo

C ▼

Créditos

12

Requerimientos de salón

Un requisito

Carreras donde se dicta

1 carrera(s) seleccionada(s)

Ingenieria Informatica X

Seleccione semestre y año ▼

Guardar cambios Eliminar asignatura

Por otro lado este es el modal de edición, el cual permite editar o eliminar algunas de las entidades importantes del sistema, se permiten editar los datos relevantes además de poder modificar futuros cambios que puedan surgir con el paso del tiempo.

9. Instalación y ejecución

Para la instalación y ejecución del proyecto se necesita ejecutar los siguientes comandos, todos ellos desde la raíz del proyecto:

1. `npm install`
2. `npm install --save-dev electron-builder`
3. `npm install xlsx`
4. `npm run build`

Con este último se crea una carpeta dist dentro de la carpeta del proyecto, dentro de esta carpeta se encuentra el instalador de la aplicación para escritorio. Por otro lado, si se quiere ir directo al ejecutable, entrar a la carpeta win-unpacked y abrir ejecutar la app de “CalendarioFIT”.

Si surgen problemas de seguridad con el control de inteligente de aplicaciones de Windows, seguir los siguientes pasos:

Ir a Windows Defender

Seleccionar → Protección contra virus y amenazas

Luego → Configuración de antivirus y protección contra amenazas (Administrar la configuración)

Una vez ahí → Exclusiones - Agregar o quitar exclusiones

Si, darle a agregar exclusión y poner la carpeta donde se crea el ejecutable, debería ser:

`C:\Users\Usuario\AppData\Roaming\calendariofit`

Esto funciona para crear un ejecutable, si se quiere ejecutar localmente en modo de dev paso a paso a seguir es:

1. `npm install`
2. `npm run build`

10. Limitaciones actuales

A pesar de cumplir con los requerimientos planteados, el sistema presenta ciertas limitaciones inherentes al alcance definido:

10.1 Uso monousuario

El sistema está diseñado para ser utilizado por una única persona en un entorno local. No soporta acceso concurrente desde múltiples dispositivos.

10.2 Base de datos local

La información se almacena en un archivo SQLite local, por lo que no existe sincronización automática ni existe respaldo automático.

10.3 Ausencia de sistema de roles

Actualmente no existe autenticación ni permisos diferenciados. Por ende, no hay trazabilidad de cambios por usuario.

10.4 Escalabilidad institucional

El sistema fue diseñado para la Facultad de Ingeniería y Tecnologías. Si bien puede adaptarse, actualmente no contempla múltiples facultades o múltiples sedes.

10.5 Gestión avanzada de conflictos

El sistema detecta conflictos básicos de horario, pero no incluye optimización automática de horarios, algoritmos de planificación avanzada o asignación automática de aulas.

11. Futuras mejoras

11.1 Aplicación multiusuario

Actualmente la aplicación de CalendarioFIT funciona como una app de escritorio con una base de datos local, pensada para que sea utilizada por un único usuario.

Como mejora a futuro se propone evolucionar el sistema a uno multiusuario permitiendo:

- Registro e inicio de sesión
- Gestión de múltiples usuarios
- Roles diferenciados
- Personalización de horarios según el usuario

Esto implicaría una implementación de autenticación segura, separar los datos por usuario y añadir un control de permisos.

11.2 Base de datos centralizada

Actualmente la aplicación cuenta como se mencionó previamente con una base de datos SQLite con almacenamiento local. Una mejora importante sería migrar hacia una base de datos centralizada en un servidor (PostgreSQL o MySQL son simplemente algunos ejemplos posibles)

Entre los beneficios que generaría esta migración se encuentran:

- Acceso desde múltiples dispositivos
- Sincronización en tiempo real
- Respaldo automático
- Mejora en la escalabilidad

Esto sin embargo requeriría crear una API backend (como por ejemplo [Node.js](#)), implementar una autenticación con tokens (JWT) y migrar el modelo de datos existentes hasta el momento.

11.3 Auditoría

Se propone incorporar un módulo de auditoría para poder llevar el registro de las acciones críticas del sistema, de manera que se puedan ver los cambios realizados y quien los realizó. Algunos ejemplos pueden ser: cambios en carreras o materias, creación, modificación o eliminación de horarios, entre otros.

Al tener una auditoría se obtendría una trazabilidad completa, una mayor transparencia, además de detectar errores o usos indebidos de manera más sencilla.

11.4 Expandir a todas las facultades

Actualmente el sistema se encuentra disponible para la mayoría de las carreras de la FIT (Facultad de Ingeniería y Tecnología), aunque al tener un modelo escalable, se desea a futuro, poder ampliar el alcance de la aplicación al resto de facultades que pertenecen a la UCU.

11.5 IA para recomendaciones de horarios

Por último, pero no menos importante, una mejora innovadora sería incorporar un sistema de recomendaciones a la hora de crear horarios basado en Inteligencia Artificial. El objetivo de la IA sería:

- Sugerir combinaciones óptimas de horarios
- Detectar conflictos de manera automática
- Priorizar materias según carga horario y tipo de materia

Esto convertiría a la app en una herramienta inteligente, que permite al usuario tener una mejor experiencia a la vez de una toma de decisiones asistida.

Alerta con 2 grupos superpuestos (si choca con un modulo deberia saltar error aunque sean materias distintas (choca micro1 y p1 mismo todo))

Visualizacion de alertas sin importar la carrera (no aparece si cambias de carrera las alertas, solo de la primera).